Lingbot-vla部署及其性能分析

1. 部署及适配

在bw22节点复用之前va的容器

pusht数据集：

```
代码块
1   lerobot-manual-download --repo-id lerobot/pusht --save-dir /workspace/datasets/
```

数据集结构

```
代码块
1   data/chunk-000/episode_000000.parquet ~ episode_000049.parquet (50 episodes)
2         videos/chunk-000/episode_000000.mp4 ~ episode_000049.mp4
3         info.json (meta)
4         stats.json
```

模型下载：

LingBot-VLA 4B 预训练 位置: /workspace/models/lingbot-vla-4b-posttrain-robotwin/

Qwen2.5-VL-3B-Instruct (tokenizer) 位置: /workspace/models/Qwen2.5-VL-3B-Instruct/

模型路径配置

```
代码块
1   model.model_path: /workspace/models/lingbot-vla-4b-posttrain-robotwin
2   model.tokenizer_path: /workspace/models/Qwen2.5-VL-3B-Instruct
```

代码&环境修改：

- config.json修改了dtype字段
- 新建configs/robot_configs/pusht.yaml & assets/norm_stats/pusht.json
- data 字段:

- datasets_type: vla
- data_name: pusht
- train_path: /workspace/datasets/pusht
- norm_type: bounds_99
- norm_stats_file: assets/norm_stats/pusht.json

- lingbotvla/data/vla_data/base_dataset.py: video_backend 从 torchcodec换成pyav
- 在flex_attention.py中添加LINGBOT_DISABLE_FLEX_COMPILE 开启融合
- /usr/local/lib/python3.10/dist-packages/torch/_inductor/kernel/flex/flex_attention.py 两处else分支中conf.block_m → getattr(conf, "block_m", getattr(conf, "block_m1", 0))
- lerobot升级为0.4.2

以上和大凯之前适配的一致

H20上一样的 启动命令

区别在于video_backend 是torchcodec

代码块

```
HF_ENDPOINT=https://hf-mirror.com \
HF_HOME=/zhangyifan/zhangyf/vla/.cache/huggingface \
HF_DATASETS_CACHE=/zhangyifan/zhangyf/vla/.cache/datasets \
PYTHONPATH=/zhangyifan/zhangyf/vla/lingbot-vla \
torchrun --nproc-per-node=8 \
  --master_port=12345 \
  -m lingbotvla.train.train \
  --config configs/vla/pusht.yaml \
  --output_dir /zhangyifan/zhangyf/vla/lingbot-vla/output_pusht
```

2. 测试

测试配置 pusht.yaml (micro_batch_size=8, global_batch=64, 8 GPU)

编译配置torch.compile(flex_attention, mode="max-autotune-no-cudagraphs")

启动命令

cd /workspace/lingbot-vla

export HIP_VISIBLE_DEVICES=0,1,2,3,4,5,6,7

export CUDA_VISIBLE_DEVICES=0,1,2,3,4,5,6,7

bash benchmark_pusht.sh configs/vla/pusht.yaml

3. 优化

训练表现：跑120步的最后50步 下表是开箱性能基础上做的基础优化 后续优化以两边都开启融合 关闭freeze_vision_encoder为准 即红色背景列

	BW1000					H20		
	开箱	优化器fused	优化器fused+全局compile	左边基础上优化triton	freeze_vision_encoder = true	H20(开箱)	H20(开启全局compile)	freeze_vision_encoder= true
s/it	2	1.951	1.281	1.198	0.958	1.404	0.899	0.73
samples/s/gpu	4	4.39	6.23	6.67	8.35	5.68	8.9	10.96
BW1000/H20	70%	71.9%(+1.9%)	110.3%(+37.4%)	117.2%(+45.3%)	147%(+36.7%)	100%	156.17%(+56.2%)	193%(+36.83%)

测试了一下 如果H20开箱的video后端和bw一样走pyav 平均每步耗时是1.56 所以这还是有影响的 但是在bw上不管是直接pip install 还是装cpu版本的都不行 如果能走torchcodec的后端 那么性能会继续提升10%

两边都抓了一下trace

Returned 224 rows in 57 ms

Add debu

name	record_count	total_duration_ms	avg_duration_ms
ncclDevKernel_ReduceScatter_Sum_f32_RING_LL(ncclDevKernelArgsStorage<4096ul>)	290	92.986099	0.320641
ncclDevKernel_AllGather_RING_LL(ncclDevKernelArgsStorage<4096ul>)	292	91.32289	0.312749
nvjet_tst_128x256_64x4_2x1_v_bz_coopA_NTN	346	85.381792	0.246768
nvjet_tst_256x128_64x4_2x1_v_bz_coopA_NNT	426	68.299196	0.160326
triton_tem_fused_slice_backward_zeros_2	72	66.954719	0.929926
nvjet_tst_256x128_64x4_1x1_v_bz_coopA_TNT	144	53.338984	0.370409
void at::native::detail::chunk_cat_cuda_kernel<float, c10::BFloat16>(c10::BFloat16**, float*, long*,...	290	36.642015	0.126351
nvjet_tst_256x128_64x4_2x1_v_bz_coopA_TNT	144	34.084572	0.236698
nvjet_tst_192x192_64x3_1x1_v_bz_coopB_NNN	256	31.987074	0.124949
nvjet_tst_256x128_64x4_1x1_v_bz_coopA_NNT	70	25.910449	0.370149
at::native::detail::split_with_sizes_copy_out_contiguous_no_cast_kernel(char**, char**, long*, long...	290	18.430345	0.063552
nvjet_tst_192x160_64x3_1x2_h_bz_coopB_NTT	128	18.167973	0.141937
nvjet_tst_144x128_64x6_1x2_h_badd_TNN	128	16.887622	0.131934
nvjet_tst_128x160_64x5_2x1_v_bz_NTT	214	12.452084	0.058187
nvjet_tst_128x160_64x5_2x1_v_bz_coopA_bias_TNT	64	9.473842	0.148028
nvjet_tst_192x192_64x4_1x1_v_bz_coopB_TNN	64	9.134127	0.14272
nvjet_tst_160x192_64x3_2x1_v_bz_coopB_NTN	64	9.097937	0.142155
nvjet_tst_256x128_64x4_1x2_h_bz_coopA_NNT	64	9.060409	0.141568
triton_tem_fused_0	72	8.593841	0.119358

H20

Returned 222 rows in 71 ms

Ad

name	record_count	total_duration_ms	avg_duration_ms
triton_tem_fused_slice_backward_transpose_view_zeros_2	72	464.823206	6.455877
ncclDevKernel_ReduceScatter_Ring_Simple_PreMulSum_f32(ncclDevKernelArgsStorage<102...	358	416.324875	1.162918
ncclDevKernel_AllGather_Ring_Simple_Sum_i8(ncclDevKernelArgsStorage<1024ul>)	220	216.484643	0.984021
Cijk_Ailk_Bjlk_BBH_MT256x256x16_SE_AMAS3_BW_DTL0_BL1_DRB16_ALT_KME1_ETSP_EP...	612	64.209591	0.104917
conv3d_ncxdhw32_bf16_wrw_implicitgemm_wg64x64_tt4x4_su32_nogroup_dhw128n8_com...	2	62.983166	31.491583
at::native::detail::split_with_sizes_copy_out_contiguous_no_cast_kernel(char**, char**, long...	290	59.263976	0.204358
void at::native::detail::chunk_cat_cuda_kernel<float, c10::BFloat16>(c10::BFloat16**, float*,...	290	57.229604	0.197343
Cijk_Ailk_Bljk_BBH_MT256x128x16_SE_AMAS3_BW_DTL0_BL1_DRB16_ALT_KME0_ETSP_EP...	400	49.605974	0.124014
triton_tem_fused_0	72	44.847748	0.622885
conv3d_ncdhw_bf16_fwd_implicitgemm_wg64x128_tt4x8_su32_group_common_gfx928	2	37.66734	18.83367
void at::native::(anonymous namespace)::multi_tensor_apply_kernel<at::native::(anonymous ...	100	33.333804	0.333338
Cijk_Alik_Bljk_BBH_MT256x256x16_SE_AMAS3_BW_DTL0_BL1_DRB16_ALT_KME0_ETSP_EP...	208	30.993053	0.149005
Cijk_Alik_Bljk_BBH_MT128x128x32_SE_AMAS3_BW_DTL0_BL1_DRB16_ALT_KME0_ETSP_EP...	74	25.937352	0.350504
Cijk_Alik_Bljk_BBH_MT256x256x16_SE_AMAS3_BW_DTL0_BL1_DRB16_ALT_KME0_ETSP_EP...	192	22.909396	0.119319
void at::native::(anonymous namespace)::multi_tensor_apply_kernel<at::native::(anonymous ...	308	18.471203	0.059971
void at::native::elementwise_kernel_manual_unroll<128, 8, at::native::gpu_kernel_impl_noca...	988	14.003003	0.014173
void at::native::(anonymous namespace)::multi_tensor_apply_kernel<at::native::(anonymous ...	308	13.869606	0.045031
Cijk_Ailk_Bljk_BBH_MT256x256x16_SE_AMAS3_BW_DTL0_BL1_DRB16_ALT_KME0_ETSP_EP...	70	13.049846	0.186426
triton_poi_fused__to_copy_add_arange_cat_cos_mul_permute_pow_reciprocal_sin_split_sub_...	144	12.687527	0.088107

BW1000

仅triton_tem_fused_slice_backward_transpose_view_zeros_2就相差了199ms 如果能抹平这一差距 那么性能将拉近 这个已经在和泽贤一块看了 另外torchcodec的问题也提了新包需求了

分析trace文件 BW的前向传播的完全没有overlap lingbot-va的解决办法是增大fsdp粒度 粒度越粗 prefetch 的数据量越大 减少all gather通信开销 但是在lingbot-vla上的粒度已经是block级别（fully_shard(layer)）

更换triton包后 性能提升（这是单次迭代的 上面俩图是两次迭代）

name	record_count	total_duration_ms	avg_duration_ms
ncclDevKernel_AllGather_Ring_Simple_Sum_i8(ncclDevKernelArgsStorage<1024ul>)	162	241.588392	1.491286
ncclDevKernel_ReduceScatter_Ring_Simple_PreMulSum_f32(ncclDevKernelArgsStorage<1024ul>)	127	142.534476	1.122318
triton_tem_fused_slice_backward_transpose_view_zeros_2	36	65.373262	1.815923
Cijk_Ailk_Bjlk_BBH_MT256x256x16_SE_AMAS3_BW_DTL0_BL1_DRB16_ALT_KME1_ETSP_EPS1_FL0...	306	32.547227	0.106363
conv3d_ncxdhw32_bf16_wrw_implicitgemm_wg64x64_tt4x4_su32_nogroup_dhw128n8_common_a...	1	31.473929	31.473929
at::native::detail::split_with_sizes_copy_out_contiguous_no_cast_kernel(char**, char**, long*, long...	145	28.653787	0.197612
void at::native::detail::chunk_cat_cuda_kernel<float, c10::BFloat16>(c10::BFloat16**, float*, long*,...	145	28.347541	0.1955
Cijk_Ailk_Bljk_BBH_MT256x128x16_SE_AMAS3_BW_DTL0_BL1_DRB16_ALT_KME0_ETSP_EPS1_FL0...	200	27.292606	0.136463
conv3d_ncdhw_bf16_fwd_implicitgemm_wg64x128_tt4x8_su32_group_common_gfx928	1	18.841605	18.841605
Cijk_Alik_Bljk_BBH_MT256x256x16_SE_AMAS3_BW_DTL0_BL1_DRB16_ALT_KME0_ETSP_EPS1_FL0...	104	15.532859	0.149354
Cijk_Alik_Bljk_BBH_MT128x128x32_SE_AMAS3_BW_DTL0_BL1_DRB16_ALT_KME0_ETSP_EPS1_FL0...	37	12.989568	0.351069
void at::native::(anonymous namespace)::multi_tensor_apply_kernel<at::native::(anonymous names...	154	12.700494	0.08247
Cijk_Alik_Bljk_BBH_MT256x256x16_SE_AMAS3_BW_DTL0_BL1_DRB16_ALT_KME0_ETSP_EPS1_FL0...	96	11.410306	0.118857
triton_tem_fused_0	36	11.151809	0.309772
void at::native::(anonymous namespace)::multi_tensor_apply_kernel<at::native::(anonymous names...	42	10.550522	0.251202
void at::native::(anonymous namespace)::multi_tensor_apply_kernel<at::native::(anonymous names...	154	8.571048	0.055656
void at::native::(anonymous namespace)::multi_tensor_apply_kernel<at::native::(anonymous names...	42	8.436601	0.200871
void at::native::(anonymous namespace)::multi_tensor_apply_kernel<at::native::(anonymous names...	36	7.715967	0.214332

triton算子单次迭代耗时缩短到H20的一倍 性能提升到6.67samples/s/gpu

3.0 绑核

bw22节点hy-smi --showtopo信息：

```
HCU[0]          : (Topology) Numa Node 3
HCU[0]          : (Topology) Numa Affinity 3
HCU[1]          : (Topology) Numa Node 1
HCU[1]          : (Topology) Numa Affinity 1
HCU[2]          : (Topology) Numa Node 1
HCU[2]          : (Topology) Numa Affinity 1
HCU[3]          : (Topology) Numa Node 0
HCU[3]          : (Topology) Numa Affinity 0
HCU[4]          : (Topology) Numa Node 7
HCU[4]          : (Topology) Numa Affinity 7
HCU[5]          : (Topology) Numa Node 5
HCU[5]          : (Topology) Numa Affinity 5
HCU[6]          : (Topology) Numa Node 5
HCU[6]          : (Topology) Numa Affinity 5
HCU[7]          : (Topology) Numa Node 4
HCU[7]          : (Topology) Numa Affinity 4
```

GPU_NUMA=0:3,1:1,2:1,3:0,4:7,5:5,6:5,7:4

3.1 FSDP通信

在lingbotvla/distributed/torch_parallelize.py中reuse-aware reshard（延后 reshard 减少重复 all-gather）+ embed_tokens 独立 shard

性能从0.98s提升到0.74s

02_fsdp2_parallelize.patch

实现原理

3.1.1原本实现

首先明确一下当前每个FSDP unit 的概念：调用 fully_shard(module) 包裹的那个模块及其参数 FSDP 把该模块的全部参数打包成一个 flat 参数组 FSDPParamGroup 在本模型中就是 torch_parallelize.py 里每层 fully_shard(layer, **fsdp_kwargs) 的一个 Qwen2_5_VLDecoderLayer(或 parent layer) 总共 74 个 unit(counter 日志 groups=74) 切分粒度由fully_shard 的位置决定 也就是lingbot-va里修改的那个

这个unit的原本行为是在每次forward前unshared 每张卡都拿到完整参数副本 然后计算forward（unshared过程）计算过程中也不是只有一次 而是根据FSDP的粒度 先最浅粒度 到深粒度 此时每个unit都持有完整参数副本 都是unshared 这是基于all gather实现的

反向过程中unit的顺序是先深粒度 再浅粒度 在此过程中就是基于reduce-scatter做梯度分片 每卡保留1/N 然后reshared释放本层的整份参数 在本地显存操作

一次完整的迭代包括前向和反向 结束后 每张卡只有shared状态

	前向	反向
Unit 粒度顺序	浅到深	深到浅
行为	unshared	reshared
通信原语	All gather	Reduce scatter

PS：这个模型中原生的粒度就是每一层的decoder layer 一共74个unit

qwenvl.model.layers 的 36 层(VLM 塔)

qwen_expert.model.layers 的 ~36 层(action 专家塔)

外加 embed_tokens、lm_head、patch_embed/merger、expert_visual 等几个独立小模块

这里实现的问题在于 这个模型是一个双塔结构

代码块

```
models = [self.qwenvl.model, self.qwen_expert.model]
1272
1273          # RMSNorm
1274          num_layers = self.qwenvl.config.num_hidden_layers # 36
1275          flex_block_mask = None
1276          flex_block_mask_shape = None
1277          for layer_idx in range(num_layers):
```

这就导致每个layer_idx 都会先做一次models[0].layers[layer_idx](hidden_states, compute_kqv=True) 两个塔各取一次qkv 然后接着又调 models[1].layers[layer_idx](hidden_states, att_output, output_atten=True) 来基于attention_interface 把两塔结果合并(交叉注意力 attend 两者)得到 att_output

反向传播中一样有两次调用 第一次调用完就触发了

代码块

```
reshard_after_backward=True
```

这会导致第二次backward use开始时发现上一次的参数已经被收走了 需要算第二份梯度 必须重新unshard(all-gather) → 再 reduce-scatter → 再 reshard所以反向传播中两次use自己昂会有重复通信

3.1.2 代码优化逻辑

代码块

```
post_forward_indices = self._post_forward_indices   # 记录本 unit 前向被使用的顺序
add = (original_reshard_after_backward
    and self._training_state.name == "PRE_BACKWARD"
    and len(post_forward_indices) > 0)               # 还有更早的 forward use 尚未 backward
if defer_reshard:
    self.reshard_after_backward = False              # 本次先不 reshard,保留 unsharded 参数
return original_post_backward(self, *unused)
```

将reshard_after_backward=True放到第二次反向传播之后 即如果还有没被消费的前向传播 就不急着reshared 直到消费完前向传播中的成果 让该参数在连续多次反向传播中使用间共享同一份unsharded 数据 只在最后一次 use 的 backward 后真正 reshard —— 剔除中间的重复 all-gather + reduce-scatter

2) embed_tokens 独立 shard (`enable_fsdp2_embed_tokens_shard: true`)

`fully_shard(embed_tokens)` 把它从"根 unit"里拆出来单独一个 FSDP unit,避免每次forward/backward 全量 all-gather 巨大的 `embed_tokens`。

- 又缩短0.06s

3.2 flex attention

lingbotvla/models/vla/pi0/flex_attention.py

03_flex_attention.patch

3.2.1 原生实现

原本都要把qkv和maskpadding到128的倍数再建立mask block 但是36 层 LLM 的 Q/K/V shape 完全一致 每次重建是浪费

代码块

```
q_len_rounded = _round_up_to_multiple(q_len, block_size)  # 向上取整到 128 的倍数
pad_q = q_len_rounded - q_len
query_states = F.pad(query_states, (0, 0, 0, pad_q), value=0.0)
```

原本的实现所有逻辑都整合在flex_attention_forward函数中 优化后解耦 单独构建函数

3.2.2 优化后实现

代码块

```
# 独立出专门的 mask 构建函数
def build_flex_block_mask(attention_mask, num_attention_heads, query_length, key_value_length):
    causal_mask = attention_mask[:, None, :query_length, :key_value_length]
    if causal_mask.shape[1] == 1 and num_attention_heads > 1:
        causal_mask = causal_mask.expand(-1, num_attention_heads, -1, -1)
```

```
    b_mask, h_mask, q_len, kv_len = causal_mask.shape
    block_size = 128

    mask_4d = create_mask(
        mod_fn=_precomputed_mask_factory(causal_mask),
        B=b_mask, H=h_mask, Q_LEN=q_len, KV_LEN=kv_len,
        device=causal_mask.device,
    )
    return create_block_mask(
        mask_mod=_precomputed_mask_factory(mask_4d),
        B=b_mask, H=h_mask, Q_LEN=q_len, KV_LEN=kv_len,
        BLOCK_SIZE=block_size,
        device=causal_mask.device,
        _compile=False,
    )

def flex_attention_forward(..., block_mask=None):
    if block_mask is None:
        block_mask = build_flex_block_mask(...)  # 延迟构建
    # 直接使用传入的 block_mask
```

然后优化后的mask在build_flex_block_mask中直接用原始长度 `create_mask` 和 `create_block_mask` 内部已经能处理非对齐的序列长度 只要传入正确的长度参数 这些函数会自动在内部处理边界情况 无需外部手动padding

而block mask的复用机制： 第一次调用

代码块

```
block_mask = build_flex_block_mask(mask, num_heads, q_len, kv_len)
```

构建block_mask 然后后续就直接复用已构建的 block_mask（零开销）

代码块

```
for step in range(num_steps):
    output = flex_attention_forward(q, k, v, mask, block_mask=block_mask)
```

总结一下就是优化后只有首次需要构建mask 并且不需要padding

也就是说之前每次的mask不管传进来的seq_len哪怕一样 都要重新构建 现在优化后 只有传进来的seq_len不一样才要重复构建 和paading配合使用的 原本实现是每次都把seq_len padding到128 但是仍然每次都重新创建 现在是不padding了 输入的长度爱多少多少 只要没见过就构建一次mask 见过了就复用之前的

lingwen/models/vla/pi0/modeling_lingbot_vla.py

04_llm_pipeline.patch

1) `image_embeds` **去同步** (L1194)

原: `split_sizes=(image_grid_thw.prod(-1)//...).tolist()` 触发 GPU→CPU 同步 + `torch.split` / `stack` 。

新: 本调用路径图像网格等长,直接 `reshape(batch, -1, dim)` ,去掉设备同步和 graph break。

2) block_mask 跨层复用 (L1274-1345)

`flex_block_mask` 在第 1 层按 shape 生成一次,shape 不变(同 batch 内 36 层一致)就复用,不再每层重建。

3.3 vision Conv3d 内存布局

文件: `lingbot/models/vla/pi0/qwen_in_vla.py`

05_vision_channels_last.patch

位置: `Qwen2_5_VisionPatchEmbed.forward` (L76-86)

把 patch 的 `view` 结果用 `memory_format=torch.channels_last_3d` 摆好后再进 `self.proj` (Conv3d),显著降低 DCU 上卷积访存。原代码是 `view(...).to(dtype)` 后直接 `proj` ,内存连续度差。

收益: 视觉 patch embed 前向明显变快(在卷积带宽受限于的 DCU 上尤为明显)。

老生常谈 不多解释 原本占比不大所以收益不大

4.最终性能

	BW1000	H20
s/it	0.7192	0.899
samples/s/gpu	11.14	8.9

BW1000/H20	125%